

G-TADS: GUI Task-Ability Decoupling Strategy for High-Adaptability Multimodal Intelligent Agents

Zhiqiang Xia, Xinyuan Zhang, Yang Li, Yuchen Liu, Runyu Shi, Jiaming Xu*

Xiaomi Corporation, Beijing, China

{xiazhiqiang5, zhangxinyuan8, liyang134, liuyuchen8, shirunyu, xujiaming1}@xiaomi.com

Abstract—Graphical User Interface (GUI) automation aims to help users interact with devices efficiently, and the integration of open-source multimodal large language models (MLLMs) with GUI agents shows great potential in this area. However, existing open-source model-based solutions typically treat GUI tasks as a single end-to-end process, overlooking the optimization of input-output design and training strategies, which results in poor performance in real-world applications. To address these issues, this paper systematically analyzes input-output strategies and finds that incorporating historical operation descriptions into the input significantly enhances task performance. Based on this, we propose the Task Ability Decoupling Strategy (G-TADS), a GUI multimodal intelligent agent framework that decomposes tasks into two independent subtasks: generating operation semantic descriptions and generating specific actions. We validated the G-TADS framework on the AITZ dataset and the Mind2Web dataset, which we extended with semantic annotations, demonstrating its strong generalization ability in multi-task scenarios. Our code is available at <https://github.com/xzq11111/G-TADS>.

Index Terms—GUI Agent, Multimodal Large Language Model (MLLM), Task Ability Decoupling

I. INTRODUCTION

Graphical User Interfaces (GUIs) are a core element of human-computer interaction, widely used in devices such as smartphones and computers. GUI automation leverages artificial intelligence (AI) technologies to perform complex operational workflows based on natural language instructions, reducing user effort. This approach not only enhances convenience for general users but also supports individuals with disabilities or limited technical skills [1]. Recent advancements in multimodal large language models (MLLMs) have catalyzed research into intelligent GUI agents that integrate linguistic and visual reasoning capabilities, demonstrating substantial application potential in understanding user intents and executing tasks efficiently [2].

State-of-the-art studies demonstrate the strong multimodal capabilities of proprietary large-scale models (e.g., GPT-4o [3]) in specific tasks. However, these models rely heavily on carefully designed prompts or additional structured data (e.g., HTML or XML) to perform effectively. In the absence of such structural information, their performance degrades significantly. Furthermore, proprietary models face limitations in deployment due to high costs and concerns about data privacy. Open-source medium-scale models (e.g., Qwen2-VL [4], MiniCPM-V [5], InternVL2-8B [6]) offer lower computational

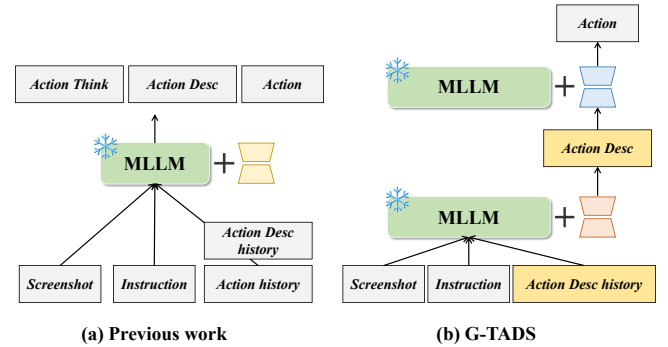


Fig. 1: Comparison of GUI task processing approaches. (a) Previous work directly generates actions, lacking a clear semantic understanding of historical operations. (b) G-TADS uses action descriptions to define historical semantics and adopts a two-stage reasoning paradigm, decoupling planning from decision-making to simplify training and reduce output pressure.

costs and greater customization, making them promising alternatives. However, these models often struggle with complex GUI tasks due to suboptimal design and training strategies.

As a representative of end-to-end models, Seeclick [7] uses GUI grounding to endow the Vision-Language Model (VLM) with the ability to predict the coordinate of operations. However, this is insufficient to understand the reasoning behind actions and the relationships between mobile apps. In contrast, [8] employs the Chain-of-Action-Thought (CoAT) method, using the screen description, the thought process of the next action, the textual description of the next action, and the possible action outcomes to provide an information basis for the agent's decision making. However, we found that not all input data is essential. Moreover, predicting semantic and action sequences concurrently within a single model escalates the difficulty of modeling.

To this end, this paper delves into two questions: what information is needed for the planning and reasoning process of the GUI, and how to model such information and predict accurate action coordinates. Therefore, we first systematically analyze the impact of putting different information into the input or output on the GUI results. Then, we propose a GUI Task Ability Decoupling Strategy (G-TADS), which decomposes GUI tasks into two independent subtasks. The model first

*Jiaming Xu is the corresponding author.

predicts the next semantic task description, based on which the operation location or action instruction is determined.

The main contributions of this paper are as follows:

- **Optimized Input-Output Design:** We systematically analyze input-output strategies and demonstrate that incorporating semantically rich historical operation descriptions significantly improves model performance.
- **Decoupled Task Design and Multimodal Fine-Tuning:** We propose G-TADS, which decouples semantic generation and specific operations into two subtasks. Through LoRA fine-tuning, we achieve efficient task learning and reasoning for open-source models.
- **Cross-Dataset Validation and Dataset Expansion:** We validate the G-TADS framework on the Android-In-The-Zoo (AITZ) dataset, achieving significant performance gains. Furthermore, we extend the Mind2Web dataset with semantic annotations. Experiments demonstrate strong generalization capabilities of G-TADS in multi-task scenarios. The extended dataset is available at https://huggingface.co/datasets/xzq11111/Mind2Web_with_act_desc.

II. RELATED WORK

A. GUI Agent

In the early stages, GUI automation tasks predominantly relied on scripts and customized rules, heavily dependent on manually crafted predefined workflows. These approaches were typically task-specific, lacking flexibility and generalization capabilities.

In recent years, the emergence of multimodal large language models (e.g., GPT-4o [3]) has revolutionized interaction paradigms. Leveraging their robust language and image understanding capabilities, GUI automation [9, 10, 11] has progressively evolved into intelligent systems where LLMs function as professional agents. Some models utilize GUI screenshots with embedded HTML/DOM or XML structures as inputs (e.g., WebDreameer [12], AppAgent-V2 [13]), while others incorporate screenshots with historical action information (e.g., SeeClick [7], AITZ [8]). However, these approaches often suffer from input redundancy, which may lead to task conflicts during training. To address these issues, this paper optimizes the input-output design and introduces the G-TADS framework, which decouples semantic understanding from action learning.

B. Multimodal Large Language Models

In the domain of Multimodal Large language models (MLLMs), proprietary models such as GPT-4V [14] and Gemini [15] exhibit exceptional multimodal foundational capabilities. Without requiring fine-tuning, these models can serve as the backbone for GUI agents (e.g., MMAC-Copilot [16]), offering generality and convenience. However, compared to open-source VLMs, their task flexibility is limited. Open-source models, by incorporating OCR and grounding capabilities (e.g., Qwen2-VL [4], MiniCPM-V [5], InternVL [6]), provide greater adaptability for GUI tasks. This flexibility is crucial

for GUI agents requiring comprehensive multimodal data understanding. We further conduct a comparative analysis of various open-source models within the G-TADS framework against their original configurations.

III. METHOD

A. Overview

This section introduces the design of GUI Agent task modeling and the proposed GUI Task Ability Decoupling Strategy (G-TADS). By optimizing input-output designs and decoupling task abilities, G-TADS enhances the performance of medium-sized open-source multimodal models in GUI tasks. The G-TADS framework divides GUI tasks into two independent subtasks: the action description generation module and the concrete action generation module. Through task ability decoupling, we effectively avoid task conflicts, improve training efficiency, and fully leverage the potential of medium-sized multimodal models. Fig. 2 illustrates the overall process of our method. In the following, we will discuss the task modeling of the GUI Agent and the design and implementation of the G-TADS framework in detail.

B. Task Modeling

To accurately model GUI tasks, we employed different input-output designs and evaluated their impact on model performance. The core objective of the task is to predict the next action based on user-provided inputs, such as the current screenshot, instructions, and historical actions. We define the following terms:

- **Current screenshot:** S_t , representing the screenshot at the current time t .
- **User instruction:** I , representing the user's desired functionality.
- **Historical action information:** H , which includes the specific locations, semantic descriptions, and other details of past actions. It can include up to the previous 4 actions.
- **Action description:** D , representing the semantic description of an action.
- **Action thinking:** T , representing what action should be taken and what outcome the selected action will lead to.
- **Specific action:** A , representing the detailed action.

Based on task requirements, we designed different input-output configurations as follows:

Figure 1. Use the current screenshot S_t and user instruction I as the input, and next specific action A_t as the output.

Figure 2. Use the current screenshot S_t , user instruction I , and historical actions $H = \{A_{t-1}, A_{t-2}, \dots\}$ as the input, and next specific action A_t as the output.

Figure 3. Use the current screenshot S_t , user instruction I , and historical action descriptions $H = \{D_{t-1}, D_{t-2}, \dots\}$ as the input, and next action description D_t and specific action A_t as the output.

Figure 4. Use the current screenshot S_t , user instruction I , and historical action descriptions $H = \{D_{t-1}, D_{t-2}, \dots\}$

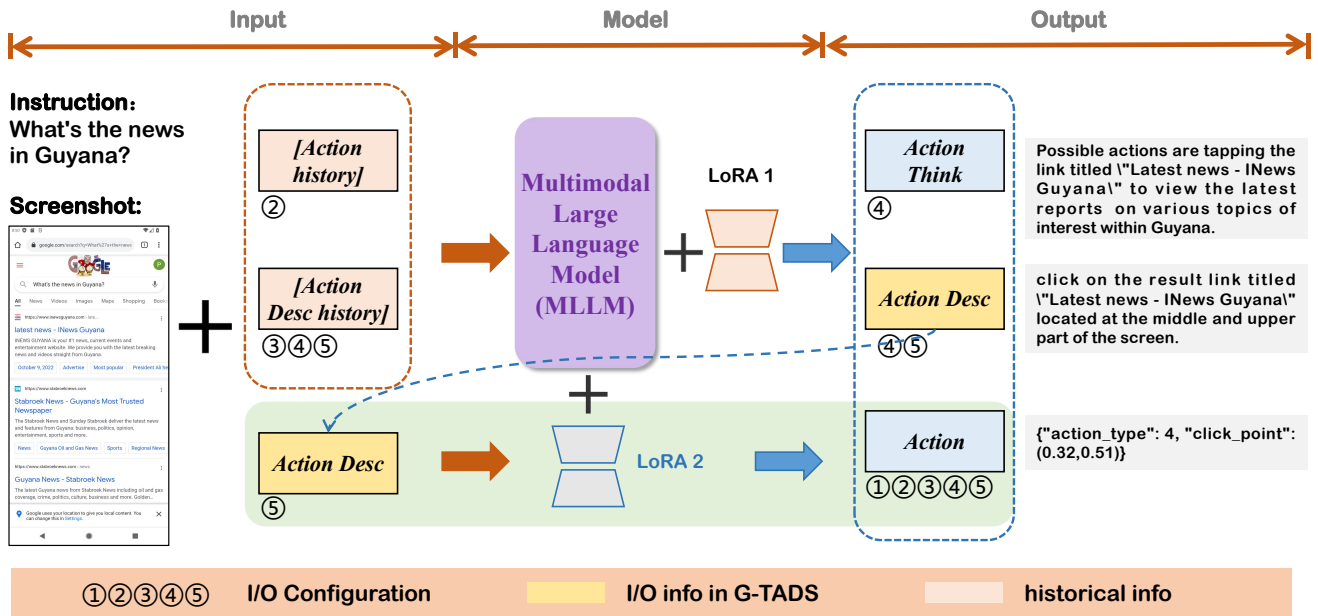


Fig. 2: Overview of GUI Task Ability Decoupling Strategy (G-TADS). The figure illustrates the two-stage approach of our framework: the first stage focuses on planning and generating action descriptions, while the second stage performs the decision-making process, incorporating the action descriptions generated in the first stage into the input, and ultimately outputs only the specific actions.

as the input, and next action thinking T_t , next action description D_t and specific action A_t as the output.

The four configurations correspond to components ①, ②, ③, and ④ in Fig. 2, respectively, providing a structured representation of the input-output settings in our experimental framework.

We conducted experiments based on the Qwen2-VL-7B model and found that configure 3 performed the best, establishing our task modeling approach.

C. GUI Task Ability Decoupling Strategy (G-TADS)

To enhance the handling capability of complex GUI tasks, we propose the G-TADS framework, which aims to decouple different abilities within the task. Component ⑤ in Fig. 2 illustrates the I/O configure of the G-TADS framework. The G-TADS framework consists of two main modules: the module for generating action semantic descriptions and the module for generating specific actions. Each module is responsible for different tasks, improving the training efficiency and inference accuracy of the model.

1) Action Semantic Description Generation Module

The task of this module is to generate the semantic description of the next action based on the input information, including the current screenshot, user instruction, and the semantic descriptions of historical actions.

2) Specific Action Generation Module

The task of this module is to generate the next specific action based on the generated semantic description of the next action, combined with the current screenshot, user instruction and the semantic descriptions of historical actions.

D. Model Training and Inference

1) Training Process

During training, we leverage an open-source multimodal large model and optimize both the action semantic description generation module and the specific action generation module through LoRA fine-tuning. By performing LoRA fine-tuning, each module focuses on its specific subtask, thereby enhancing the model's performance on GUI tasks.

2) Inference Process

During inference, the base model is first loaded, followed by the first LoRA module, which generates the semantic description of the next action. Then, the second LoRA module is switched on, which references the semantic description generated by the first LoRA module to infer the next specific action.

This inference process decouples the different abilities within the GUI task by sequentially combining the two LoRA fine-tuned modules, allowing each module to focus on its respective objective. As a result, the inference efficiency and performance of the model are improved. Additionally, the LoRA fine-tuning method ensures that model parameters are efficiently adjusted under resource constraints to meet the specific task requirements.

IV. EXPERIMENTS

In this section, we validate the effectiveness of the G-TADS framework through a series of experiments that cover different models and datasets.

TABLE I: The experimental results of different input-output strategies in the AITZ dataset.

Input			Output		SCROLL	CLICK		TYPE		PRESS	STOP	TOTAL	
his_act	his_act_des	his_img	act	act_des	match	class	match	class	match	match	match	class	match
			✓		33.22	91.19	66.93	77.64	77.64	46.30	70.34	84.04	62.41
✓			✓		46.59	86.73	61.73	92.60	92.60	64.49	64.29	85.99	63.51
	✓		✓	✓	56.91	78.25	57.86	88.40	88.40	61.10	89.09	83.83	64.54
✓	✓		✓	✓	50.42	79.14	58.00	90.78	90.78	63.19	88.10	83.43	64.06
		✓	✓		51.91	59.17	36.73	74.40	74.40	71.28	91.07	73.98	51.16
✓		✓	✓		57.40	65.79	40.17	86.95	86.95	60.05	65.87	77.99	51.59
	✓	✓	✓	✓	47.68	75.85	50.72	81.98	81.98	55.26	78.27	78.68	55.46
✓	✓	✓	✓	✓	64.56	65.13	40.24	79.11	79.11	68.93	88.29	79.57	54.70

* Each input-output configuration includes the current screenshot and user instructions in the input. *his_act*, *his_act_des*, *his_img*, *act*, and *act_des* represent history action, history action description, history image, action, and action description, respectively. The results highlight the impact of incorporating historical semantics and descriptions on task performance across different input-output strategies.

A. Datasets

1) Android-In-The-Zoo

The AITZ [8] dataset extends the AITW [17] dataset through two stages. In the first stage, mismatched instruction-action sequences are removed via targeted sampling and manual validation. In the second stage, GPT-4V [14] generates semantic information (screen description, action inference, next action, and result summary), which is verified by human experts for consistency. The distribution is shown in Table II.

TABLE II: The number of operations for different types in the AITZ

	SCROLL	CLICK	TYPE	PRESS	STOP	Total
Train	1723	7729	1459	1010	1998	13919
Test	601	2736	500	383	504	4724

2) Mind2Web

Mind2Web [18] is the first dataset specifically designed for developing general-purpose web agents. Building on an existing dataset and following the annotation method used in AITZ, we utilized GPT-4o [3] to generate semantic descriptions for each operation. This resulted in a high-quality dataset comprising over 2,000 tasks. Our dataset is available for open access on Hugging Face.

B. Implemental Details

We conducted experiments in a distributed environment with 8 NVIDIA A800 GPUs, following SeeClick’s hyperparameters: LoRA rank of 64, batch size of 4, gradient accumulation of 8, learning rate of $3e-5$, and 10 epochs. The ViT module was fully fine-tuned, and LoRA (Low-Rank Adaptation) was used to optimize specific LLM modules, excluding [‘lm_head’, ‘embed_tokens’], minimizing interference and ensuring targeted optimization of key modules for improved performance and stability.

C. Evaluation Metrics

The evaluation metrics for the AITZ dataset experiment follow those from SeeClick [7], chain-of-action agents [19] and Cogagent [9]. We use two main metrics to evaluate

model performance: operation type action-matching score and complete operation detail action-matching score. The complete operation detail match requires all aspects, such as scroll direction, input text, click position, and pressed buttons, to match the target exactly.

The evaluation metrics for the Mind2Web [18] dataset experiment follow SeeClick and Mind2Web. We calculate element accuracy (ele_acc), operation F1 (op_F1), and step success rate (step_SR) to evaluate the model.

D. Ablation Study

We explored the performance of different I/O configurations based on the Qwen2-VL-7B model on the AITZ dataset. The action-matching score for configurations 1, 2, and 3 were 62.41%, 63.51%, and 64.54%, respectively, while configuration 4 achieved only 56.82%. These results suggest that including excessive COAT information in the output of AITZ makes it difficult for the model to learn specific actions. Therefore, we conducted an ablation experiment for configurations 1, 2, and 3, where the input included both specific actions and semantic descriptions, and the output contained the semantic description of the next action as well as the specific action. As shown in Table I, we also conducted a comprehensive comparison experiment by incorporating historical screenshots into the input.

By comparing the first four rows with the last four rows in Table I, it can be observed that the action-matching scores of the first four rows consistently outperform those of the last four rows at the same positions. This indicates that incorporating historical screenshots in the input degrades the model’s performance, likely due to the lack of explicit operational information in the screenshots, which introduces redundant noise.

Next, we analyzed the performance of the first four rows. For the first row, when the input included only the current screenshot and user instructions, the action-matching score was the lowest, suggesting that the inclusion of historical information effectively introduces valuable context. When *his_act* and *his_act_des* were separately included, the action-matching scores improved, confirming the previous conclusion that historical information provides richer context for

TABLE III: Performance Comparison Between the G-TADS Framework and the Baseline Model on the AITZ Dataset.

	SCROLL	CLICK		TYPE		PRESS	STOP	TOTAL	
	match	class	match	class	match	match	match	class	match
Qwen2-VL-7B (base)	50.42	79.14	58.00	90.78	90.78	63.19	88.10	83.83	64.54
Qwen2-VL-7B (G-TADS)	54.24	86.44	63.85	87.6	87.6	55.09	80.95	86.62	66.19
MiniCPM-V2.6-8B (base)	58.74	82.60	61.26	86.2	86.2	57.18	76.39	85.88	64.80
MiniCPM-V2.6 (G-TADS)	57.07	84.36	62.98	84.4	84.4	50.13	84.92	85.84	65.77
InternVL2-8B (base)	69.72	70.61	51.39	88.8	88.8	71.80	80.16	83.81	62.34
InternVL2-8B (G-TADS)	64.73	73.50	52.71	87.0	87.0	66.32	87.90	84.19	62.68

the model, aiding in understanding operational intent and enhancing prediction accuracy. We also observed that when the input included only *his_act_des*, the performance was better than when only *his_act* were included. However, when both *his_act* and *his_act_des* were included together, the action-matching score was lower than when only historical actions were included. This suggests that while the inclusion of historical information is beneficial, an excess of such information may introduce noise and reduce model performance, particularly because *his_act* and *his_act_des* contain overlapping information, with the latter typically providing more detailed context.

In summary, through ablation experiments with various input-output combinations, we have verified that when the input includes historical action descriptions *his_act_des*, and the output contains both action descriptions and specific actions, achieved the highest action-matching score.

E. Framework Evaluation

To further enhance model performance, we incorporated GPT-4o generated semantic descriptions of the next action into the input, allowing the model to predict specific actions based on these descriptions. With this design, action type prediction accuracy reached 99.85%, and the accuracy of complete action-matching score increased to 94.48%. Specifically, for Click actions, the type prediction accuracy was 99.98%, but the exact match rate for click position was only 90.68%, leaving room for improvement. Therefore, we propose the G-TADS framework, which decouples the task into two sub-tasks and optimizes them separately.

We validated the G-TADS framework by conducting experiments with three models of similar parameter scales: Qwen2-VL-7B, MiniCPM-V2.6-8B, and InternVL2-8B. We compared the performance of the optimal input-output configurations (base model) with that of the G-TADS framework. As shown in Table III, the G-TADS framework significantly outperformed the base model on the action-matching score across the entire dataset, demonstrating its advantage in complex task modeling. Results from different models further validate the framework’s generality and robustness, confirming its effectiveness and applicability.

F. Comparison with State-of-the-art

In this section, we compare the performance of fine-tuning methods based on open-source models on the AITZ dataset.

We first replicated the SeeClick approach using Qwen-VL, as shown in Table IV, but its performance was suboptimal. This can be attributed to the limited performance of the base model and the lack of comprehensive semantic descriptions in the input history, which only included coordinates of past actions, leading to significant information loss.

The results of the Auto-UI (with CoAT) method are from the AITZ paper. This method relies on externally generated action summaries, which are impractical in real-world applications. We removed this external input and fine-tuned the Qwen2-VL-7B model to output action thinking and descriptions, followed by the generation of specific actions in a CoAT-like manner. However, the experimental results showed subpar performance, likely due to excessive output information, which increased the difficulty of learning the specific actions.

Additionally, to mitigate the impact of the base model, we also conducted experiments using the best-performing method from the AITZ paper with Qwen2-VL-7B. While the ShowUI method in Table IV achieved relatively accurate predictions of action types, its poor click localization ability led to a low action-match score. In contrast, our method achieved the optimal action-matching score.

TABLE IV: Comparison of Methods on AITZ.

Method	class	match
SeeClick(Qwen-VL based) [7]	83.90	59.22
Auto-UI (with CoAT) [8]	82.98	47.69
Qwen2-VL-7B(with AT AD) [8]	80.04	56.82
ShowUI [20]	93.5	60.3
G-TADS-7B	86.62	66.19

G. Generalization on Mind2Web

In this section, we extend the G-TADS framework to the Mind2Web dataset to evaluate its performance in more complex task scenarios. Specifically, we compare the G-TADS framework with baseline models across three test sets: Cross-Task, Cross-Website, and Cross-Domain, using three evaluation metrics. As shown in Table V, the G-TADS framework outperforms the baseline model on nearly all test sets and metrics. This result demonstrates the effectiveness of G-TADS in complex task modeling, as its task decoupling optimization enhances both the model’s understanding of operation semantics and its precision in executing specific

TABLE V: Performance Comparison Between the G-TADS Framework and the Baseline Model on the Mind2Web Dataset.

	Cross-Task			Cross-Website			Cross-Domain		
	ele_acc	op_F1	step_SR	ele_acc	op_F1	step_SR	ele_acc	op_F1	step_SR
Qwen2-VL-7B (base)	17.43	89.71	15.66	18.09	83.93	14.66	16.04	86.02	14.41
Qwen2-VL-7B (G-TADS)	18.29	89.59	16.78	20.88	84.40	17.35	16.92	87.03	15.17
MiniCPM-V2.6-8B (base)	16.68	86.22	14.73	18.03	82.23	14.64	16.24	87.04	14.25
MiniCPM-V2.6 (G-TADS)	17.18	87.97	15.10	18.41	84.28	15.47	16.38	87.14	14.70
InternVL2-8B (base)	31.92	86.24	27.30	29.85	82.52	22.89	29.98	85.96	27.07
InternVL2-8B (G-TADS)	32.96	87.18	28.16	31.20	83.92	24.84	30.07	86.04	26.74

actions. These experiments further validate the adaptability and generalization ability of G-TADS across diverse tasks, websites, and domains, highlighting its potential as a universal multimodal task modeling solution.

V. CONCLUSION

In conclusion, this paper explores the optimal input-output strategies and demonstrates that incorporating historical action descriptions into the input significantly improves model performance. Based on this finding, we propose the GUI Task Ability Decoupling Strategy (G-TADS), which exhibits strong generalization capabilities. We first validate the effectiveness of G-TADS on the AITZ dataset, where the experimental results show that, in multi-task scenarios, G-TADS outperforms existing fine-tuning methods based on similarly sized open-source models, achieving significant performance improvements. Next, we extend the Mind2Web dataset with semantic annotations, and G-TADS also performs excellently on this dataset. Given the limitations of current datasets, future work will focus on constructing more comprehensive datasets that cover a wider range of scenarios, as well as designing more complete test sets and evaluation standards to better assess the performance of the G-TADS framework and other methods.

REFERENCES

- [1] Biao Wu et al. “Foundations and recent trends in multimodal mobile agents: A survey”. In: *arXiv preprint arXiv:2411.02006* (2024).
- [2] Chaoyun Zhang et al. “Large Language Model-Brained GUI Agents: A Survey”. In: *arXiv preprint arXiv:2411.18279* (2024).
- [3] Aaron Hurst et al. “Gpt-4o system card”. In: *arXiv preprint arXiv:2410.21276* (2024).
- [4] Peng Wang et al. “Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution”. In: *arXiv preprint arXiv:2409.12191* (2024).
- [5] Yuan Yao et al. “Minicpm-v: A gpt-4v level mllm on your phone”. In: *arXiv preprint arXiv:2408.01800* (2024).
- [6] Zhe Chen et al. “Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 24185–24198.
- [7] Kanzhi Cheng et al. “Seeclck: Harnessing gui grounding for advanced visual gui agents”. In: *arXiv preprint arXiv:2401.10935* (2024).
- [8] Jiwen Zhang et al. “Android in the zoo: Chain-of-action-thought for gui agents”. In: *arXiv preprint arXiv:2403.02713* (2024).
- [9] Wenyi Hong et al. “Cogagent: A visual language model for gui agents”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 14281–14290.
- [10] Junyang Wang et al. “Mobile-Agent-v2: Mobile Device Operation Assistant with Effective Navigation via Multi-Agent Collaboration”. In: *arXiv preprint arXiv:2406.01014* (2024).
- [11] Yadong Lu et al. “Omniparser for pure vision based gui agent”. In: *arXiv preprint arXiv:2408.00203* (2024).
- [12] Danijar Hafner et al. “Mastering diverse domains through world models”. In: *arXiv preprint arXiv:2301.04104* (2023).
- [13] Yanda Li et al. “Appagent v2: Advanced agent for flexible mobile interactions”. In: *arXiv preprint arXiv:2408.11824* (2024).
- [14] Josh Achiam et al. “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [15] Gemini Team et al. “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context”. In: *arXiv preprint arXiv:2403.05530* (2024).
- [16] Zirui Song et al. “Mmac-copilot: Multi-modal agent collaboration operating system copilot”. In: *arXiv preprint arXiv:2404.18074* (2024).
- [17] Christopher Rawles et al. “Androidinthewild: A large-scale dataset for android device control”. In: *Advances in Neural Information Processing Systems* 36 (2024).
- [18] Xiang Deng et al. “Mind2web: Towards a generalist agent for the web”. In: *Advances in Neural Information Processing Systems* 36 (2024).
- [19] Zhuosheng Zhang and Aston Zhang. “You only look at screens: Multimodal chain-of-action agents”. In: *arXiv preprint arXiv:2309.11436* (2023).
- [20] Kevin Qinghong Lin et al. “ShowUI: One Vision-Language-Action Model for GUI Visual Agent”. In: *arXiv preprint arXiv:2411.17465* (2024).